Day 7: Bash Scripting and IP Sweep Basics

Today, I focused on understanding bash scripting at a beginner level and how it helps me automate commands I normally type manually in the terminal. My goal was to take a simple task like pinging an IP address, save the output to a file, filter the output using pipes, and then use that same idea inside a basic IP sweep script.

Understanding Bash Scripting

Bash scripting (the way I'm learning it) is the process of writing a sequence of terminal commands inside a text file. Instead of repeating the same commands again and again, I can put them into a script and run them whenever I need. That is why these files are often used to automate repetitive tasks.

Saving a Ping Result into a Text File

One of the first commands I practiced was ping. If I want to send only one ICMP packet, I use the -c option with the value 1. I learned that -c means count (how many ping requests I want to send).

- **-c** means **count** (how many ping requests I want to send)
- **1** means send **one** ping

Example:

```
ping 1.1.1.1 -c 1 > IP.txt
```

The > symbol redirects the output into a file instead of printing it on the screen. So this command saves the ping result into IP.txt.

To view the file content:

```
cat IP.txt
```

Filtering a Specific Result Using Pipes and grep

If I want to see only a specific thing inside a file, I can combine commands using a pipe (|). I learned that the pipe takes the output of the first command and uses it as the input for the second command.

For example, I can use grep to search for a pattern in the output. I used "64 bytes" because this text usually appears when ping succeeds, which tells me the host is reachable and responding.

```
cat IP.txt | grep "64 bytes"
```

Extracting Part of the Output Using cut

After grep filters the line I want, I can send that output into cut. I understood cut as a way to extract a specific part from the result.

cut uses a delimiter to split the text into fields. The delimiter is chosen with -d, and the field number is chosen with -f.

- **-d** sets the **delimiter** (the character used to split the text)
- **-f** selects the **field number** you want

When the delimiter is a space, cut basically counts words separated by spaces. For example, -f 4 extracts the 4th word.

```
cat IP.txt | grep "64 bytes" | cut -d " " -f 4
```

Cleaning the Output Using tr

Then I used tr. I learned tr can be used for deleting characters, replacing characters, and changing letter cases. In my case, I used it to delete a character from the output.

I used tr -d ":" to remove the colon (:) from the result.

```
cat IP.txt | grep "64 bytes" | cut -d " " -f 4 | tr -d ":"
```

Creating the Bash Script File

To create the script, I used Mousepad because I liked working with a GUI text editor. I created the file with a .sh extension.

```
mousepad IPsweep.sh
```

At the top of the script, I wrote the shebang line. I learned that this line tells Linux that the file should be executed using bash.

```
#!/bin/bash
```

Passing an IP Address and Checking for Missing Input

When I run the script, I use ./IPsweep.sh followed by the IP address. The first value after the script name is stored in $1.

```
./IPsweep.sh <IP address>
```

So $1 represents the IP address the user provides. I used an if statement to check if $1 is empty, which means the user forgot to enter the IP.

```
if [ "$1" == "" ]
then
echo "You forgot an IP address"
echo "Syntax: ./IPsweep.sh <IP address>"
```

Scanning with a Loop (IP Sweep)

If the user does provide an IP address, the script continues with else. Then I used a loop to go through a range of hosts using seq. I learned that seq stands for sequence, so seq 1 254 counts from 1 to 254.

In the loop, I ping each address using $1.$IP, then filter with grep, then cut the part I want, then clean it with tr.

I also added & at the end so that the pings run in the background together, which makes the scan faster. done ends the loop, and fi ends the if statement.

```
else
for IP in $(seq 1 254);
do
ping -c 1 $1.$IP | grep "64 bytes" | cut -d " " -f 4 | tr -d ":" &
done
fi
```

This day helped me connect a few Linux basics together: redirecting output to a file, filtering output with pipes, and then combining those ideas inside a simple bash script. I’m still learning, but writing it step by step helps me understand what each command is doing.

PROOF ON MEDUIM https://medium.com/@mohamd2001momani/day-7-learning-bash-scripting-and-building-a-simple-ip-sweep-a502007073af